

SCC.351: Advanced Cyber Security

Lab 2: ECC

Task 1: Adding Signing to Your Voting Server

In this task, you will build upon task 2 of lab 1. You will create a voting registrar that generates and distributes ECDSA signing key pairs to eligible voters. The voting client will be modified to sign votes before posting to the voting server, and the voting server will be modified to verify the signature before including the submitted vote in the result.

Your modified system will consist of three components:

- **Registrar:** generates key pairs for eligible voters, distributes signing keys and stores public verification keys for server access.
- **Client:** signs votes using the voter's ECDSA private key, submits signed votes and requests results.
- **Server:** receives and validates votes, verifies signatures, counts votes and returns results on request.

Question 1

Modify your data flow diagram from lab 1 to incorporate the registrar.

Question 2

Write a script for the registrar using the skeleton file provided on Moodle (`registrar.py`). Your implementation must support the following:

- Generate an ECDSA key pair for n voters, where n is provided as user input, using the `cryptography.io` library available at: <https://cryptography.io/en/latest/hazmat/primitives/asymmetric/ec/#>. Each key pair must be linked to a different voter ID.
- Store the public keys in a public key database (e.g., `public_keys.json`) in PEM format.
- Save private keys in PEM format.

Question 3

Modify your `server.py` and `client.py` files from lab 1 as follows.

Your `client.py` file should:

- On input of a voter ID, retrieve the voter's private key from the registrar.
- Sign the vote using the ECDSA signature algorithm.
- Send the voter ID, vote and signature to the server.

Your `server.py` file should:

- Retrieve the voter's public key from the registrar's database.
- Verify the signature. If valid, record the vote and return the result if queried.
- Return an error if the signature is invalid.

Question 4

Test that your solution handles incorrect input to the voting client, and that the voting server rejects any ballots that are accidentally or maliciously modified.

Your voting client should return an informative error if:

- The voter inputs an invalid voter ID.
- The voter inputs an invalid vote option.

The voting server should reject ballots (and return an error message) if:

- The signature has been modified.
- The voter ID included in the ballot has been modified.
- The candidate included in the ballot has been modified.

In these events, the signature should be invalid! Why?

Additionally, the voting server should reject ballots (and return an error message) if any of the following fields are missing: `voter ID`, `candidate` or `signature`.

Task 2: STRIDE Threat Modelling

Consider the STRIDE threat model, introduced in Lecture 2. Reflect on the changes you have made to the voting system in task 1, specifically the use of ECDSA signatures.

Which categories of STRIDE are addressed by adding ECDSA signatures to the voting protocol?

For each relevant category, explain ONE threat that is addressed and how ECDSA helps to mitigate the associated threat. In your answer, assume that the registrar securely distributes private signing keys to each individual voter (i.e., only the voter and registrar will know the voter's signing key).

You may find it useful to consider the following prompts:

- How does the server know that the vote came from the correct voter?
- What happens if someone tried to change the vote content?
- Can a voter deny having submitted a vote?

Use the table below to prepare your answer.

STRIDE category	Description of threat	How ECDSA mitigates the threat